Performance Engineering (2360013) — Assignment 1

Student: חביב ראמי · ID 325420180 **Instructor:** Nadav Amit · Spring 2026 **Environment:** provided VM `course-08` — KVM guest, Intel Xeon Gold 5420+, 4 vCPUs, kernel 7.0.0-15-generic, ext4, 7.7 GiB RAM. Kernel sources at `/usr/src/linux-7.0.0`.

How this investigation was run

Each phenomenon was driven as a hypothesis loop: establish a baseline **with run-to-run variance**, predict before measuring, find the single measurement that separates competing hypotheses, **rule out alternatives with data**, and **confirm the mechanism against the kernel source under `/usr/src`**. Raw tool output for every probe is filed under `task1/` and `task2/` (referenced inline by filename); the full timestamped trail is in the git history.

Results at a glance

- **Task 1 — pipe latency improves under CPU load.** Cause: **scheduler wake-placement**. With idle CPUs available, `select_idle_sibling()` spreads the two pipe partners onto *different* cores, making every round-trip a cross-core wakeup (~11 µs); background load removes the idle CPUs, so the pair is co-located on one core (~5 µs). Frequency (DVFS) and idle/HLT cost were ruled out with data; confirmed at `kernel/sched/fair.c:select_idle_sibling`.
- **Task 2 — v2 (4 MiB prepare) reads faster.** Cause: **prepare block size sets page-cache folio order** — v2 builds 2 MiB PMD-order folios, v1 stays at 16 KiB. The 8–9 % is the **dTLB win from PMD-mapping** those folios (measured here at +12.7–20.2 % via hugetlb). On this VM the file-cache PMD mapping is structurally disabled (`CONFIG_READ_ONLY_THP_FOR_FS` unset), so only ~1 % reproduces. Fragmentation ruled out (zero disk I/O); confirmed in `mm/readahead.c`, `mm/memory.c`, and the kernel config.

Task 1 — Why pipe latency *improves* under background CPU load

Phenomenon. `lat_pipe` (lmbench "hot-potato" Unix-pipe round-trip) reports ~11 µs idle, but ~5 µs when `stress-ng --cpu 3` runs alongside it — a ~2× *speed-up* from adding load.

Result (one line). The cause is **scheduler wake-placement**, not frequency or idle/halt cost. When idle CPUs exist, the scheduler spreads the two pipe partners onto *different* cores, so every round-trip pays a cross-core wakeup (reschedule IPI + cache-line bounce). Background load removes the idle CPUs, so the scheduler co-locates the pair on one core, turning each round-trip into a cheap same-core context switch. Confirmed in `kernel/sched/fair.c`, `select_idle_sibling()` line 110 (`select_idle_cpu`).

---

1. Environment (as shipped — recorded before changing anything)

`task1/env_asshipped.txt`

- **KVM full-virt guest** (`systemd-detect-virt = kvm`), Intel Xeon Gold 5420+, **4 vCPUs**, 1 thread/core, each vCPU presented as its own socket; 1 NUMA node. Kernel 7.0.0-15-generic.
- **No cpufreq driver** in the guest (`cpupower frequency-info`: "no or unknown cpufreq driver active"; governors Not Available). `cpu MHz` fixed at 2000.
- **No cpuidle states** (`cpupower idle-info`: driver `none`, "No idle states").
- `perf_event_paranoid = -1` (full perf access incl. kernel).

Methodological note (lecturer slide 65 vs our setup). The deck recommends pinning the governor to `performance` and disabling idle states for a stable benchmark. Those knobs **do not exist inside this guest**, so we instead (a) recorded the environment as shipped, (b) reproduced the effect, then (c) controlled one variable at a time, and verified frequency was constant *via counters* rather than by pinning. This is a reasoned deviation, not an oversight.

2. Baseline + variance

`logs/task1_20260610_1427.log` (lat_pipe prints to stderr; numbers are from the session log)

condition	n	median	min–max
no load	10	**11.12 µs**	9.88 – 11.20
`stress-ng --cpu 3`	10	**5.26 µs**	4.74 – 6.39

Effect = **2.1× (5.9 µs)**, far larger than the run-to-run spread. The effect is real and stable.

3. Three hypotheses, and how each was settled

We compared the good (loaded, fast) and bad (idle, slow) cases and drilled down from counters to controlled experiments to source.

F — CPU frequency scaling (DVFS). RULED OUT.

`task1/p2_perfstat_{noload,load}.txt`

`perf stat` frequency, from `cycles ÷ ref-cycles` (a ratio, so immune to the fact that `lat_pipe` self-calibrated to different iteration counts under contention):

	cycles	ref-cycles	ratio = freq/nominal
no load	1.620 B	1.203 B	**1.347**
with load	73.485 B	54.567 B	**1.347**

Identical → when the core actually executes, it runs at the same ~2.7 GHz in both conditions. Frequency does not change. **F is out.** (The raw `perf stat` latency of ~69 µs is measurement overhead and is *not* compared across runs — only the ratio is.)

I — Idle / HLT→VMEXIT wakeup cost. RULED OUT.

`task1/p2b_pin2core_idle.txt`, `task1/p2b_pin2core_busy.txt`

Initial (wrong) reading: pinning *both* pipe ends to one core reproduced the speed-up (`p2_pin1core_noload.txt`: 5.42 µs, no load) — suggesting idle removal was the cause. But that probe changed **two** variables at once (no-idle *and* same-core). A placement-controlled experiment (Phase 2B) fixed placement = cross-core and flipped only idle:

placement	cores idle between turns?	latency
cross-core (cpus 0,1)	yes	11.2 µs
cross-core (cpus 0,1) + a spinner pinned to each	**no**	10.9 µs

Keeping the cores busy with placement held cross-core did **not** help. **I is out.** (Confound handled: the no-spinner case (no contention) and the spinner case (contention) are *both* ~11 µs, so the slowness is the cross-core placement, not CPU contention.)

P — Scheduler wake-placement. RULED IN.

`task1/p2_pin1core_noload.txt` + Phase 2B + Phase 4 evidence

Holding the *other* variable (idle) fixed and flipping placement:

placement	core state	latency
same core (pin 1)	busy	**5.4 µs**
cross core (pin 2, idle or busy)	either	~11 µs
loaded, unpinned (scheduler chose)	busy	5.3 µs
idle, unpinned (scheduler chose)	idle	11.1 µs

Same-core ⇒ fast, cross-core ⇒ slow, independent of idle/busy. **Placement is the axis.**

4. Direct evidence (histogram + sampled callgraph)

Off-CPU latency histogram (bpftrace on `sched_switch`) — `task1/p4_offcpu_hist_*.txt`. Time each pipe end stays blocked per handoff:

```
no load:   mode [8, 16) µs   (139,094 hits)      with load:  mode [2, 4) µs  (311,579 hits)
           [16,32) µs 33,289                                   [4, 8) µs  40,707
```

The whole distribution drops ~4×, matching the 11→5 µs latency.

Sampled callgraph (`perf record -e sched:sched_switch -g`) — `task1/p4_callgraph_noload.txt`:

```
16.63%  lat_pipe S ==> swapper/0:0     ← end #1 blocks; cpu0 goes idle
14.74%  lat_pipe S ==> swapper/3:0     ← end #2 blocks; cpu3 goes idle
        anon_pipe_read → schedule → __schedule
```

No-load, the two ends sit on **separate cores (0 and 3)** and each blocks to the idle task (`swapper`) — i.e. the pair is spread, and every resume is a cross-core wakeup. This is the mechanism made visible.

(The per-CPU switch-count files `p2b_latpipe_cpudist_*` are aggregated over many iterations and are inconclusive about momentary co-location; treated as weak supporting data only.)

5. Kernel-source confirmation — `kernel/sched/fair.c`

`task1/p4_select_task_rq_fair.txt` (fair.c:8579), `task1/p4_select_idle_sibling.txt` (fair.c:7836)

Wakeup placement path: `try_to_wake_up → select_task_rq → select_task_rq_fair`. For a normal task wakeup it takes the **fast path**, `select_task_rq_fair` line 64:

```
new_cpu = select_idle_sibling(p, prev_cpu, new_cpu);
```

Inside `select_idle_sibling`, the early returns use `target` / `prev` *only if already idle* (lines 24–26, 31–37). The decisive step is **line 110**:

```
i = select_idle_cpu(p, sd, has_idle_core, target);   /* scan LLC domain for ANY idle CPU */
if ((unsigned)i < nr_cpumask_bits)
        return i;
```

- **Idle system (no bg load):** `select_idle_cpu` finds an idle CPU and returns it → the woken pipe partner is placed on a *different* core than the waker → the pair is split across cores → each hot-potato round-trip is a cross-core wakeup (reschedule IPI + pipe-buffer/`task_struct` cache-line bounce between cores) ≈ **11 µs**.
- **Loaded system:** `select_idle_cpu` finds **no** idle CPU (falls past line 111) → the function returns `prev` / `target` → the partner is **co-located with the waker** → same-core context switch, no IPI, no cross-core cache traffic ≈ **5 µs**.

So background load helps **by accident**: it removes the idle CPUs that `select_idle_cpu` would otherwise hand the wakee, forcing co-location.

6. Conclusion

The pipe benchmark is faster under CPU load because the Linux scheduler's idle-CPU-seeking wakeup placement (`select_idle_sibling` / `select_idle_cpu`) **spreads** the two communicating processes across cores when CPUs are idle, making every round-trip a cross-core wakeup; load removes the idle CPUs and forces same-core co-location, which is ~2× cheaper. Frequency scaling (F) was ruled out by a

constant `cycles/ref-cycles` ratio; idle/HLT cost (I) was ruled out because cross-core-but-busy stayed slow. The mechanism was confirmed by a placement-controlled experiment that toggles the effect, an off-CPU latency histogram, a sampled callgraph, and the kernel source.

Evidence index

File	Evidence
`task1/env_asshipped.txt`	environment (KVM, no cpufreq/cpuidle)
`logs/task1_20260610_1427.log`	baseline ×10 each condition
`task1/p2_perfstat_{noload,load}.txt`	F ruled out (freq ratio constant)
`task1/p2_pin1core_noload.txt`	same-core = fast
`task1/p2b_pin2core_{idle,busy}.txt`	I ruled out (cross-core busy still slow)
`task1/p4_offcpu_hist_{noload,load}.txt`	off-CPU latency histogram
`task1/p4_callgraph_noload.txt`	sampled callgraph (pair on separate cores → swapper)
`task1/p4_select_idle_sibling.txt`, `p4_select_task_rq_fair.txt`	kernel source

Task 2 — Root cause of the v1/v2 random-read gap (prepare block size)

Setup. Same 64 MiB file built two ways — v1 with default (~16 KiB) prepare writes, v2 with 4 MiB writes — then the identical random-read (`mmap`, 4 KiB, 5 s) benchmark on each. Reference: v2 ~8–9 % faster.

Result (one line). The prepare block size sets the **page-cache folio order**: v2's 4 MiB writes build the entire file out of **2 MiB (PMD-order) folios** (proven by a folio-order histogram); v1's small writes cap at 16 KiB. The 8–9 % comes from **PMD-mapping** those huge folios, which removes the dTLB misses that dominate this benchmark — a lever **measured here at +12.7 % to +20.2 %** (via `hugetlb`), bracketing the gap. On this VM the *file-cache* PMD mapping is **structurally disabled** (`CONFIG_READ_ONLY_THP_FOR_FS` not set ⇒ `do_set_pmd` never fires ⇒ `FilePmdMapped=0`), so v2's huge folios are mapped with 4 KiB PTEs — leaving only a ~1 % fault-overhead edge and gating off the dTLB term. The gap is decomposed, its mechanism measured, and its absence pinned to one kernel config.

---

1. Environment [task2/p0_env.txt]

KVM guest, kernel 7.0.0-15, **$HOME on ext4** (`/dev/vda1`). RAM 7.7 GiB (irrelevant — 64 MiB caches fully). Runtime THP = `[always]`; sysbench 1.0.20.

2. Baseline — the gap did not reproduce [task2/p1_baseline_gap.txt]

	v1 reads/s	**v2 reads/s**
median (n=5)	1,933,791	1,931,441

v2 is *not* ~8–9 % faster here (within noise). The investigation became: **why is the gap absent, and what is its real mechanism?** (You cannot investigate an effect you cannot reproduce.)

3. Rule out the obvious wrong answer — disk fragmentation [p1b_filefrag, p2_perfstat_*]

- `filefrag`: v1 = **2 extents**, v2 = **1** — trivial.
- `perf stat`: **`page-faults == minor-faults` exactly** (v1 1813=1813, v2 821=821) → **zero major faults** → the file is 100 % page-cached, **no disk I/O during the run**.
- A run that never touches disk cannot care about disk layout. **Fragmentation ruled out, with data.**

4. The mechanism — prepare block size → page-cache folio order [p5_folio_hist_*, p3_folios_*]

Direct folio-order histogram (bpftrace on `mm_filemap_add_to_page_cache`, field `order`), built during prepare:

	folio orders built	meaning
v1 (4 KiB writes)	4096 × **order-2 (16 KiB)** + 3027 × order-0	small folios only
v2 (4 MiB writes)	**32 × order-9 (2 MiB / PMD)** = whole file + 2870 × order-0	**PMD-order folios**

v2's 4 MiB writes build the entire 64 MiB file as 2 MiB PMD-order folios; v1 caps at 16 KiB. (Earlier I inferred from average folio counts that both stayed sub-PMD — the histogram disproves that; v2 clearly reaches PMD order. Lesson: count the distribution, don't average it.) The larger folios also make fault-around more effective ⇒ v2 has **2× fewer page-faults** (821 vs 1813; **13× fewer at 1 GiB**: 1301 vs 17173 [p3_bigfile.txt]) — a real but small (~1 %) edge.

5. Why the *throughput* gap is absent — the huge folios are not PMD-mapped [p2_smaps_, *p2_faultcount*_]

v2 has 2 MiB folios, but the gap needs them **mapped as 2 MiB PMD entries** so the TLB benefits: - The benchmark is **dTLB-bound**: `dTLB-load-misses` ≈ **8.74 M / 9.4 M reads ≈ 0.93 miss/read** (4 KiB TLB reach ~4 MiB ≪ 64 MiB working set). Only PMD mapping (2 MiB → 64 MiB in ~32 TLB entries) removes this. - But `FilePmdMapped = 0 kB` for both, and `do_set_pmd` **never fired** (bpftrace) → v2's 2 MiB folios are mapped with **4 KiB PTEs** → `dTLB-load-misses` **identical** v1 vs v2 (8.74 M vs 8.75 M; 7.41 M vs 7.41 M at 1 GiB) → identical cycles → only the ~1 % fault term survives. - Holds across **all THP modes** (always/madvise/never: v2 ~1 % > v1, no jump) and at **1 GiB** [p3_thp_sweep, p3_bigfile].

Decomposition of the documented 8–9 %: a small folio/fault term (~1 %, reproduced here) + a dTLB term that requires PMD *mapping* of v2's huge folios — which this kernel does not do.

5b. The dTLB lever, measured directly [p5c_force_thp.txt]

To prove the dTLB term is real (not just argued), the same random 4 KiB read pattern was run over an anonymous mapping backed by **2 MiB pages** (`MAP_HUGETLB`, PMD-mapped) vs base 4 KiB pages:

working set	mapping	dTLB-load-misses	reads/s
256 MiB	4 KiB	194,681,566	62,759,989
256 MiB	**2 MiB (hugetlb)**	**2,753** (~70,000× fewer)	**70,738,072 (+12.7 %)**
1 GiB	4 KiB	198,810,736	56,452,905
1 GiB	**2 MiB (hugetlb)**	**13,578**	**67,850,882 (+20.2 %)**

PMD mapping cuts dTLB misses to near-zero and lifts throughput **+12.7 % (256 MiB) to +20.2 % (1 GiB)** — larger at the bigger working set (the TLB-reach signature), and **bracketing the reference 8–9 %**. This is the dTLB-win magnitude on this exact CPU: the gap is structurally a huge-page/dTLB effect, and 2 MiB mapping does deliver it here. (Anonymous `MADV_COLLAPSE`/THP did *not* engage on this kernel — `EINVAL`, `AnonHugePages=0` even at THP=always; a kernel quirk, so `hugetlb` was used as the reliable toggle.) What v2 lacks is not the folios (it has 2 MiB ones) nor the hardware payoff (proven here) — it is the *file-cache* PMD mapping, gated off by config (§6).

6. Kernel-source confirmation [p4_page_cache_ra_order, p4_do_set_pmd, p4_kconfig_thp]

- **Folio order follows the I/O size** — `mm/readahead.c:467 page_cache_ra_order()`: `c new_order = min(mapping_max_folio_order(mapping), new_order); new_order = min_t(unsigned int, new_order, ilog2(ra->size)); /* ← order bounded by request size */` v2's 4 MiB writes permit order-9 (2 MiB); v1's small writes force ≤ order-2. Explains §4.
- **PMD mapping is a separate, gated step** — `mm/memory.c:5408 do_set_pmd()` installs a 2 MiB PMD only for a PMD-sized aligned folio; it **never fired** here (bpftrace) ⇒ FilePmdMapped=0.
- **The decisive gate** — `/boot/config-7.0.0-15-generic`: `CONFIG_TRANSPARENT_HUGEPAGE=y # CONFIG_READ_ONLY_THP_FOR_FS is not set ← file-backed THP for regular FS NOT compiled in` With `READ_ONLY_THP_FOR_FS` off, an mmap'd **ext4** read can **never** get a PMD file mapping, even when (as for v2) the underlying folio is already PMD-sized. So `FilePmdMapped=0` is **structural**, the dTLB win is unreachable, and the 8–9 % gap is **impossible on this kernel** — regardless of THP mode or file size.

7. Conclusion

Root cause: **the prepare write block size sets the page-cache folio order** — v2's 4 MiB writes build 2 MiB PMD-order folios (proven by the order histogram), v1's stay at 16 KiB. The *throughput* gap is the **dTLB win from PMD-mapping** those huge folios, which dominates this dTLB-bound benchmark. On this VM that mapping is structurally disabled (`CONFIG_READ_ONLY_THP_FOR_FS` unset, `do_set_pmd` never reached), so v2's huge folios are 4 KiB-mapped, dTLB is unchanged, and only the ~1 % fault-overhead term reproduces. Fragmentation was ruled out (zero disk I/O). The reference environment must enable file-backed THP, which PMD-maps v2's folios and yields the 8–9 %.

The chain, fully evidenced: v2's huge folios exist (§4 histogram) → PMD mapping of such a working set is worth +12.7–20.2 % on this CPU (§5b, measured via hugetlb) → but the file-cache PMD mapping never forms (FilePmdMapped=0, do_set_pmd never fires) → because `CONFIG_READ_ONLY_THP_FOR_FS` is unset (§6). Each link is measured or read from source; none is assumed.

Evidence index

File	Evidence
`task2/p0_env.txt`	environment (ext4, THP=always)
`task2/p1_baseline_gap.txt`	gap absent (v1≈v2)
`task2/p1b_filefrag.txt`	fragmentation trivial (2 vs 1 extent)
`task2/p2_perfstat_{v1,v2}.txt`	page-faults 2× fewer (v2); dTLB identical; 100 % minor
`task2/p2_smaps_*`, `p2_faultcount_*`	FilePmdMapped=0; do_set_pmd never fires
`task2/p5_folio_hist_{v1,v2}.txt`	**folio-order histogram: v2 = 32× 2 MiB PMD folios; v1 = 16 KiB**
`task2/p3_folios_*`, `p3_thp_sweep.txt`, `p3_bigfile.txt`	fault scaling; ~1 % edge in all THP modes
`task2/p4_page_cache_ra_order.txt`	folio order bounded by I/O size
`task2/p4_do_set_pmd.txt`	PMD file-map path (gated)
`task2/p4_kconfig_thp.txt`	`CONFIG_READ_ONLY_THP_FOR_FS` not set — the structural gate
`task2/p5_thp_dtlb_test.txt`	first dTLB microbench (inconclusive: anon THP didn't engage)
`task2/p5c_force_thp.txt`	**dTLB lever measured: 2 MiB hugetlb mapping → dTLB ~70,000× fewer, +12.7–20.2 %**